

Memoria de Práctica: Navegación Autónoma y Órbita de Obstáculos

Vehículos Autónomos

16 de abril de 2026

1. Introducción y Objetivos

La práctica se centra en la integración avanzada del *Multi-Ranger Deck*, el cual dota al dron de sensores de distancia láser de tiempo de vuelo (ToF) en cinco direcciones: adelante, atrás, izquierda, derecha y arriba [?]. Para complementar esta percepción y garantizar una navegación robusta, se integra el *Flow Deck v2*. Este último es fundamental para el éxito de la misión, ya que proporciona estabilidad mediante flujo óptico y cubre la medición de distancia hacia abajo (la única dirección no cubierta por el Multi-Ranger), permitiendo un control de altura preciso y evitando derivas laterales durante maniobras complejas [?, ?].

El objetivo principal es programar una lógica de control reactivo que permita al dron navegar de forma autónoma, detectando obstáculos y ejecutando maniobras de circunnavegación precisas.

Los hitos específicos del proyecto se definen de la siguiente manera:

- **Hito 1: Navegación y Detección:** Implementación de un avance lineal autónomo con detección de obstáculos desconocidos en la trayectoria frontal.
- **Hito 2: Maniobra de Transición:** Ejecución de un giro coordinado para lograr un posicionamiento lateral relativo al objeto detectado.
- **Hito 3: Control de Órbita Circular:** Mantenimiento de una trayectoria de 360° alrededor del obstáculo, utilizando un control proporcional para estabilizar la distancia lateral constante.
- **Hito 4: Estabilidad y Aterrizaje:** Garantizar la estabilidad del sistema durante toda la misión mediante la fusión de datos de ambos *decks* y proceder a un aterrizaje seguro tras completar la circunferencia.

2. Configuración del Sistema y Hardware

El sistema se basa en la combinación de periféricos y librerías que permiten una percepción completa del entorno y estabilidad en el vuelo autónomo.

2.1. Hardware Utilizado

La plataforma física se compone de los siguientes elementos:

- **Crazyflie 2.1:** Plataforma de vuelo base de alta agilidad.
- **Multi-Ranger Deck:** Equipa al dron con sensores de distancia láser de tiempo de vuelo (ToF) que le permiten detectar objetos en cinco direcciones: adelante, atrás, izquierda, derecha y arriba[cite: 6]. Esta configuración proporciona una percepción tridimensional del entorno[cite: 7].
- **Flow Deck v2:** La inclusión del *Flow Deck* es fundamental para mejorar la estabilidad y precisión del vuelo, combinando la información de ambos sistemas para una navegación más robusta[cite: 10]. Aporta dos ventajas técnicas principales:
 1. **Complemento de Percepción:** El Multi-Ranger no cubre la dirección inferior. El Flow Deck aporta la medición de distancia hacia abajo, completando el volumen de detección y permitiendo el control preciso de la altura.
 2. **Estabilidad Horizontal:** Mediante su sensor de flujo óptico, permite que el dron mantenga su posición X-Y, limitando las derivas durante las maniobras complejas.

2.2. Librerías y Entorno de Software

El desarrollo del algoritmo se realizó mediante un script en Python basado en la librería `cflib`[cite: 12]. Las clases y utilidades fundamentales empleadas para el control son:

- **SyncCrazyflie:** Gestiona la conexión y el ciclo de vida del dron[cite: 14].
- **MotionCommander:** Interfaz de alto nivel responsable de los comandos de vuelo relativos (avanzar, girar, etc.)[cite: 16].
- **Multiranger:** Obtiene y gestiona las mediciones de distancia síncronas proporcionadas por el deck Multi-Ranger[cite: 18].

3. Metodología y Descripción de la Tarea

El desarrollo de la solución técnica se fundamenta en la adaptación y extensión del script base proporcionado en el enunciado de la práctica. El código original ha sido modificado significativamente para transitar de un comportamiento de seguimiento de muros a uno de circunnavegación completa, estructurando la lógica de control en tres fases secuenciales. El objetivo final es que el dron busque un objeto, se oriente tangencialmente a él y lo orbite manteniendo una distancia radial constante mediante el ajuste dinámico de su tasa de rotación (*yaw*).

3.1. Fase 1: Aproximación y Detección

El dron inicia la misión avanzando linealmente a una velocidad constante (`FORWARD_SPEED` = 0.15 m/s) y emplea el sensor frontal para detectar objetos a una distancia inferior a 0.3 metros (`THRESHOLD_FRONT`). Para asegurar la robustez de esta fase, se implementó la función auxiliar `safe_read`[cite: 41, 45]. Esta función enmascara las lecturas nulas

(None)—habituales cuando el sensor no detecta nada—asignándoles un valor artificialmente alto para evitar que se active prematuramente la lógica de evasión o cercanía.

3.2. Fase 2: Posicionamiento Inicial

Tras la detección exitosa del objeto, el dron detiene por completo su avance lineal. A fin de preparar la trayectoria orbital, ejecuta un giro estático de 90° sobre su eje vertical hacia la izquierda. Esta maniobra posiciona el obstáculo directamente en el campo de visión del sensor lateral derecho, permitiendo que el control proporcional tome el relevo en la siguiente fase.

3.3. Fase 3: Órbita Circular y Control de Finalización

La fase final consiste en un bucle continuo en el que el dron avanza tangencialmente a 0.15 m/s mientras regula dinámicamente su velocidad de rotación (*yaw_rate*). Este ajuste se realiza a través de un controlador proporcional (P) que evalúa el error entre la distancia lateral objetivo (`THRESHOLD_SIDE` = 0.3 m) y la medición real obtenida por el sensor derecho (`dist_right`):

$$\text{error_side} = \text{THRESHOLD_SIDE} - \text{dist_right} \quad (1)$$

$$\text{yaw_rate} = \text{error_side} \times \text{KP_YAW} \quad (2)$$

El dron combina el comando de avance lineal con la rotación calculada sobre su eje (`rate_yaw=yaw_rate`), describiendo así una trayectoria curva ininterrumpida alrededor del obstáculo. Para garantizar la estabilidad del vuelo y evitar comandos extremos, la velocidad de rotación resultante se limitó a valores seguros utilizando la función `np.clip`.

Además, para que el sistema sea capaz de reconocer de forma autónoma cuándo ha completado la vuelta, se ha implementado un mecanismo de **acumulación de giro**. En cada iteración del bucle, el programa multiplica la velocidad de rotación actual por el periodo de muestreo (`LOOP_PERIOD`) para obtener el ángulo infinitesimal recorrido:

$$\theta_{total} = \sum (\text{yaw_rate} \times \text{LOOP_PERIOD}) \quad (3)$$

Cuando el valor absoluto de este sumatorio alcanza o supera los 360° , el dron identifica que la circunferencia ha sido completada, sale del bucle de control y procede a realizar un aterrizaje seguro y controlado.

4. Análisis de Resultados y Dificultades

Durante las pruebas experimentales para conseguir una órbita circular suave, surgieron diferentes desafíos técnicos.

4.1. Dificultades en el Ajuste de la Ganancia (K_p)

Uno de los mayores desafíos fue la sintonización empírica de la ganancia `KP_YAW` (cuyo valor final se fijó en 25).

- **K_p elevado (Sobre-corrección):** Generaba correcciones demasiado agresivas. El dron realizaba giros bruscos que provocaban inestabilidad, oscilaciones constantes (zigzag), dificultando una trayectoria fluida alrededor del objeto y llegando incluso a rotar de más y perder el objeto de su campo de visión.
- **K_p reducido (Sub-corrección):** El giro no era lo suficientemente rápido para compensar el avance lineal. Esto causaba que el dron perdiera la referencia del objeto. Al dejar de detectarlo a su derecha, el algoritmo entraba en un estado donde el dron comenzaba a girar sobre sí mismo, dando vueltas en el mismo sitio intentando buscar un obstáculo que ya había quedado atrás.

4.2. Análisis de la Geometría del Obstáculo y Errores de Trayectoria

La validación se realizó empleando a una persona como obstáculo, lo cual introdujo variables complejas que no ocurren con objetos cilíndricos perfectos (como columnas):

- **Irregularidad Morfológica (Sección Achatada):** Al utilizar un obstáculo humano, se comprobó la dificultad añadida por nuestra anatomía, que no es un cilindro perfecto, sino que presenta una sección transversal más elíptica o achatada. Esta asimetría implica que la distancia varía considerablemente dependiendo del ángulo desde el que se esté orbitando. Debido a esto, existen segmentos específicos de la órbita donde resulta mucho más difícil mantener la trayectoria correcta sin perder la referencia del sensor. Estas variaciones continuas en la distancia leída exigen una respuesta dinámica impecable, lo que implica que este reto morfológico requiere una precisión mucho mayor en el ajuste empírico de la ganancia K_p , buscando el punto exacto que permita reaccionar a la asimetría sin caer en la sobre-corrección inestable.
- **Efecto Espiral durante la Sintonización:** Durante el proceso iterativo de ajuste de la constante, sobretodo con valores de K_p bajos, se observó inicialmente un comportamiento en el que la trayectoria tendía a abrirse en una espiral de radio creciente. Esta deriva se debía a que la inercia del movimiento continuo hacia adelante, combinada con valores subóptimos del control puramente proporcional, no lograba compensar el desplazamiento lateral a tiempo. Sin embargo, una vez afinado correctamente el valor de K_p , este efecto se mitigó de manera efectiva, permitiendo que la tasa de rotación (*yaw*) cerrara la curva de forma precisa sobre el punto de inicio.

5. Conclusión

A pesar de las dificultades técnicas relacionadas con la sintonización de ganancias y la naturaleza imprevisible e irregular de los obstáculos humanos, el algoritmo desarrollado cumple exitosamente con el objetivo propuesto. Se logró programar una ruta compleja en la que el Crazyflie 2.1 avanza autónomamente, detecta un obstáculo y completa una circunferencia de 360 grados a su alrededor antes de proceder al aterrizaje seguro. Las pruebas evidencian que el equilibrio entre la velocidad tangencial y la corrección proporcional del *yaw* es la clave para una navegación robusta que permita cumplir con el objetivo.